

Projeto pessoal

Explorando o processamento assíncrono do ESP32

Sistema de sinalização com ESP32, LEDs, botão e OLED SSD1306

Relatório técnico resumido

Autor: [Sergio Cordero Calvimontes](#)
Plataforma de simulação: Wokwi
Linguagem: MicroPython
Placa-alvo: Espressif ESP32-DevKitC V4

17 de agosto de 2026

Sumário

1	Introdução	2
1.1	Controle documental desta revisão	2
2	Requisitos e definições do projeto	2
2.1	Requisitos funcionais e estados do sistema	2
2.2	Definições de hardware	4
2.3	Entrada do botão e antirrepique	6
2.4	Interface dos três mostradores	6
3	Desenvolvimento do programa <code>main.py</code>	7
3.1	Estrutura geral	7
3.2	Assincronismo cooperativo	8
3.3	O que o escalonamento cooperativo garante – e o que não garante	8
3.4	LEDs piscantes e controle de velocidade	9
3.5	Botão principal, LED verde e console de registro	9
3.6	Indicadores de atividade: barramento e escalonador	9
3.7	Mostradores de uso de CPU e RAM (dois OLEDs)	10
3.8	Console de registro colorido (TFT)	11
4	Estratégia de verificação e testes isolados	11
5	Circuito simulado	13
6	Conclusão	13
	Referências eletrônicas	14

1 Introdução

Este é um projeto pessoal que explora as capacidades de processamento assíncrono (**asyncio**) do microcontrolador ESP32. A proposta reúne conteúdos de instrumentação, eletrônica e lógica de programação por meio da simulação de um sistema embarcado baseado em ESP32: seis LEDs azuis piscando de forma assíncrona e independente entre si, um botão pulsador – um tipo de interruptor que atua apenas enquanto está sendo acionado – associado a um LED verde, e três mostradores (*displays*) atualizados concorrentemente: dois OLEDs controlados pelo circuito integrado SSD1306, exibindo em tempo real o uso de CPU e de memória RAM, e uma tela TFT ILI9341 funcionando como um console de registro (*log*) colorido.

A simulação executável está publicada no Wokwi em <https://wokwi.com/projects/471528241540407297>. O código-fonte, o diagrama do circuito, a documentação e o histórico de versões estão disponíveis no repositório GitHub em <https://github.com/Argus-Kepheus/esp32-asyncio>.

O Wokwi foi adotado como plataforma principal por oferecer suporte nativo ao ESP32 com MicroPython, edição visual do circuito, arquivo de conexão `diagram.json`, monitor serial e compartilhamento por endereço eletrônico. O repositório GitHub complementa a simulação ao fornecer rastreabilidade, controle de versões e uma estrutura adequada ao desenvolvimento no Visual Studio Code.

1.1 Controle documental desta revisão

Este relatório descreve uma revisão específica e datada do projeto, identificada pelos itens abaixo – não um estado “final”: o projeto continua sujeito a evolução, e mudanças futuras no código, no circuito ou nos testes podem tornar trechos deste texto desatualizados até a próxima revisão do relatório. “Validação”, nesta revisão, significa consistência verificada entre código-fonte, circuito e texto – não a reexecução de toda a simulação Wokwi ou de todos os testes de `tests/` nesta data (ver Seção 4 para o que foi de fato executado e quando).

Commit do Git examinado:	872d096 (branch <code>main</code>)
Data desta revisão:	17 de agosto de 2026
Placa-alvo:	Espressif ESP32-DevKitC V4 (<code>board-esp32-devkit-c-v4</code>)
Firmware MicroPython (local):	ESP32_GENERIC-20260406-v1.28.0.bin ¹
Projeto Wokwi:	https://wokwi.com/projects/471528241540407297

¹Nome de arquivo declarado no comentário de `wokwi.toml`, onde o binário é baixado manualmente e renomeado para `firmware.bin` antes do uso local. SHA-256 do binário efetivamente presente neste repositório: `cd7820d02c35d34dd403b44263129c6a511b350aea8446c229890753fe240784` – calculado localmente nesta revisão, não conferido de forma independente contra o servidor de download do MicroPython. Qualquer reprodução futura deve conferir esse hash antes de assumir que obteve exatamente o mesmo *build*.

2 Requisitos e definições do projeto

2.1 Requisitos funcionais e estados do sistema

O sistema satisfaz seus requisitos funcionais por meio de treze fluxos concorrentes sob um único escalonador cooperativo (**asyncio**): seis tarefas de LED (uma por LED azul), duas tarefas de mostrador (uma por OLED), uma tarefa de status serial, uma tarefa indicadora de escalonador ocioso, e três fluxos de monitoramento de botão – o do botão principal, aguardado diretamente por `main()`, e os dois dos botões de velocidade, despachados com `asyncio.create_task()` (Seção 3.1). O indicador de barramento ocioso

(LED laranja) não tem uma tarefa própria: é alternado de forma síncrona, por chamada de função direta, a partir de qualquer tarefa que esteja escrevendo em um dos três mostradores naquele instante (Seção 3.6); a TFT, pelo mesmo motivo, também não tem uma tarefa de atualização própria – recebe escritas síncronas disparadas a partir de várias tarefas diferentes, não de um laço periódico dedicado.

Seis LEDs azuis, seis tarefas independentes: Cada um dos seis LEDs (GPIO 26, 14, 27, 25, 33 e 12) alterna seu próprio estado em uma tarefa `asyncio` própria, logicamente independente das demais: nenhuma delas chama ou espera diretamente por outra. Isso não as isola de atrasos – sob um único escalonador cooperativo, uma escrita síncrona longa em qualquer mostrador (Seção 3.8) ainda atrasa a retomada de todas as demais tarefas, LEDs piscantes inclusive, até que essa escrita retorne (Seção 3.3). Todos compartilham um mesmo intervalo, ajustável em tempo real por dois botões dedicados (GPIO 34 diminui, GPIO 35 aumenta) que escalam o intervalo de todos os seis simultaneamente pelo mesmo fator. Na teoria, isso mantém os seis sincronizados entre si; na prática, essa sincronia é apenas nominal. Por serem tarefas independentes, cada uma mede sua própria pausa a partir do instante em que retoma a execução, e não a partir de um relógio absoluto compartilhado – pequenos atrasos causados por outras tarefas (sobretudo as escritas bloqueantes nos mostradores) se acumulam de forma diferente em cada LED a cada ciclo. Após alguns minutos de operação contínua, torna-se perceptível a dessincronização visual entre os seis, independentemente do intervalo configurado no momento. Essa dessincronização não é um defeito de implementação, e sim uma limitação inerente ao escalonamento cooperativo: nada garante que tarefas nominalmente idênticas permaneçam em fase entre si indefinidamente.

Botão e LED verde: Lê o botão conectado ao GPIO 17, com antirrepique por software, e reflete o estado estável no LED verde (GPIO 4): **solto** desliga o LED, **pressionado** liga o LED. Cada transição é registrada no console da tela TFT, em verde. Pelo mesmo motivo da dessincronização acima, a amostragem do botão compete por tempo de processamento com as demais tarefas assíncronas; em alguns instantes – tipicamente quando uma escrita bloqueante nos mostradores está em andamento – é necessário manter o botão pressionado por mais tempo que o nominal para que a janela de estabilidade de 30 [ms] do antirrepique seja efetivamente acumulada, já que ciclos de amostragem podem ser adiados enquanto o processador está ocupado.

LED laranja – barramento ocioso: Fica aceso por padrão e apaga apenas durante as escritas bloqueantes nos mostradores (`show()` / `fill_rect()` / `text()`) – lido ao contrário, apagado significa que o barramento I²C ou SPI está, naquele instante, efetivamente ocupado transferindo dados (Seção 3.6).

LED amarelo – atividade do escalonador: Alterna a cada iteração de uma tarefa dedicada que cede o controle imediatamente (`await asyncio.sleep_ms(0)`). Não é um indicador de que nenhuma outra tarefa tem trabalho pendente, nem uma tarefa de prioridade mínima real – o `asyncio` do MicroPython não tem níveis de prioridade –, mas sim uma visualização grosseira de com que frequência essa tarefa consegue retomar a execução (Seção 3.6).

Mostrador de uso de CPU (primeiro OLED): Atualizado a cada 250 [ms] no mínimo (Seção 3.3) com a fração da janela de amostragem ocupada pelas chamadas síncronas instrumentadas de desenho e transferência dos três mostradores – um substituto parcial e aproximado para “uso de CPU”, detalhado na Seção 3.7, já que o MicroPython no ESP32 não expõe essa métrica diretamente.

Mostrador de uso de RAM (segundo OLED): Atualizado a cada 250 [ms] no mínimo com a fração ocupada do heap gerenciado pelo coletor de lixo do MicroPython (`gc.mem_alloc()` / `gc.mem_free()`), não da RAM física total do ESP32 (Seção 3.7), em um barramento I²C independente do primeiro OLED.

Console de registro (tela TFT): Recebe, de forma assíncrona e a partir de várias tarefas diferentes, linhas de texto coloridas sobre os eventos do sistema – uma cor por subsistema – rolando pela tela como um registro (*log*) de atividade.

A atualização periódica dos três mostradores não serve apenas para exibir informação: sua função também é manter o processador deliberadamente ocupado, evitando que fique ocioso por longos períodos. É justamente essa carga que torna visíveis, através dos LEDs laranja e amarelo descritos acima, os instantes reais de ocupação e ociosidade do sistema – e é essa mesma disputa por tempo de processamento que explica tanto a dessincronização dos LEDs azuis quanto a eventual necessidade de segurar o botão por mais tempo.

2.2 Definições de hardware

A placa selecionada foi a **Espressif ESP32-DevKitC V4**, representada no Wokwi pelo componente `board-esp32-devkit-c-v4`. A especificação original exige apenas “MicroPython para ESP32”; portanto, a escolha da placa não decorre de uma necessidade exclusiva de desempenho. Ela foi assumida por ser uma placa oficial da Espressif, documentada, identificada de forma inequívoca no simulador e capaz de disponibilizar todos os GPIOs requeridos. A Tabela 1 apresenta a pinagem específica adotada para este projeto.

Para eventual montagem física, recomenda-se uma ESP32-DevKitC V4 equipada com módulo ESP32-WROOM, preferencialmente ESP32-WROOM-32E. Versões com módulo WROVER devem ser evitadas sem revisão do projeto, pois GPIO 16 e GPIO 17 podem ser reservados para a memória PSRAM.

Além da placa selecionada, outras placas de desenvolvimento poderiam ser utilizadas sem problemas de compatibilidade de pinos, desde que baseadas no chip ESP32 (arquitetura Xtensa LX6) com módulo WROOM – e não WROVER, S2, S3, C3 ou C6, cujos mapeamentos de GPIO, reservas de memória e conjunto de periféricos diferem do exigido por este projeto. Entre as alternativas compatíveis estão:

- Espressif ESP32-DevKitC V2, a revisão anterior da placa adotada, com o mesmo módulo ESP32-WROOM-32 e cabeçalho (*header*) de 38 pinos;
- placas genéricas “NodeMCU-32S” (ESP-WROOM-32), amplamente disponíveis de diversos fabricantes, com layout de 38 pinos equivalente;
- “DOIT ESP32 DevKit V1”, com módulo ESP32-WROOM-32 em cabeçalho de 30 pinos, desde que os GPIOs 26, 4, 16, 17 e 32 estejam expostos;
- Clones com módulo WROOM-32 de fabricantes como AZ-Delivery ou HiLetgo, que replicam o mesmo mapeamento de GPIOs do módulo original.

Em todos os casos, é necessário confirmar que os GPIOs 16 e 17 não estão reservados para PSRAM ou outra função interna do módulo específico antes de reutilizar o mapeamento de pinos deste projeto.

O primeiro LED piscante (GPIO 26) e o relógio I²C do primeiro OLED (GPIO 32) foram realocados dos GPIO 2 e GPIO 25, originalmente fixados antes do início do desenvolvimento, a pedido explícito do usuário, por motivos de layout da placa à medida que mais periféricos foram adicionados (ver o registro de decisões em `docs/EN/technical-specification.md`, §16). O GPIO 2 liberado foi posteriormente reaproveitado pelo LED indicador amarelo do escalonador, e o GPIO 25 pelo quarto LED piscante – a Tabela 1 reflete a fiação atual, não a atribuição original.

A numeração utilizada corresponde aos números dos GPIOs do microcontrolador e não à posição sequencial dos terminais físicos da placa; a coluna “Terminal físico” indica exatamente essa posição sequencial, para referência ao montar o circuito fisicamente. Todos os LEDs possuem resistores limitadores de 220 [Ω], e todos os componentes compartilham a mesma referência de terra.

Tabela 1: Pinagem completa do projeto, agrupada por subsistema.

Função	GPIO	Terminal físico ¹	Identificador no código
<i>LEDs piscantes – seis tarefas independentes, todos azuis (#0000FF) no circuito atual²</i>			
LED piscante 1	26	J2-10	red_led / RED_LED_PIN
LED piscante 2	14	J2-12	blue_led / BLUE_LED_PIN
LED piscante 3	27	J2-11	yellow_led / YELLOW_LED_PIN
LED piscante 4	25	J2-9	white_led / WHITE_LED_PIN
LED piscante 5	33	J2-8	orange_led / ORANGE_LED_PIN
LED piscante 6	12	J2-13	red_led_2 / RED_LED_2_PIN
<i>Botão principal e LED verde</i>			
Botão pulsador principal	17	J3-11	push_button / BUTTON_PIN
LED verde (indicador do botão)	4	J3-13	green_led / GREEN_LED_PIN
<i>Botões de controle de velocidade</i>			
Botão – diminuir intervalo	34	J2-5	decrease_speed_button
Botão – aumentar intervalo	35	J2-6	increase_speed_button
<i>LEDs indicadores de estado do sistema</i>			
LED laranja – barramento ocioso ⁴	13	J2-15	bus_idle_led
LED amarelo – escalonador ocioso	2	J3-15	scheduler_idle_led
<i>Primeiro OLED – mostrador de uso de CPU, barramento I²C(0)</i>			
Relógio I ² C (SCL)	32	J2-7	OLED_SCL_PIN
Dados I ² C (SDA)	16	J3-12	OLED_SDA_PIN
<i>Segundo OLED – mostrador de uso de RAM, barramento I²C(1) independente</i>			
Relógio I ² C (SCL)	15	J3-16	OLED2_SCL_PIN
Dados I ² C (SDA)	22	J3-3	OLED2_SDA_PIN
<i>Tela TFT – console de registro, SPI genuíno de 4 fios</i>			
Relógio SPI (SCK)	18	J3-9	TFT_SCK_PIN
Dados de saída SPI (MOSI)	23	J3-2	TFT_MOSI_PIN
Seleção de circuito (CS)	5	J3-10	TFT_CS_PIN
Dado/comando (D/C)	21	J3-6	TFT_DC_PIN
Reinicialização (RST)	19	J3-8	TFT_RST_PIN
Chave deslizante de modo de gravação ³	0	J3-14	–

¹Identificação do conector (J2 ou J3) e da posição do terminal no cabeçalho físico de 38 pinos da ESP32-DevKitC V4, conforme [docs/EN/hardware-reference.md](#).

²Os identificadores de código (`red_led`, `yellow_led`, `white_led`, `orange_led`, `red_led_2`) são nomes herdados de uma fase anterior do projeto, quando cada LED tinha uma cor física distinta planejada. No circuito atual todos os seis LEDs piscantes são fisicamente azuis; o nome de cada variável no código serve apenas para diferenciá-la das demais, não para descrever a cor real do componente.

³Lida apenas pelo *bootloader* ROM do ESP32 antes de qualquer script MicroPython iniciar; nenhum código de `main.py` interage com ela.

⁴Aceso por padrão (barramento ocioso); apaga durante uma escrita bloqueante em qualquer um dos três mostradores (Seção 3.6). Lido ao contrário, apagado equivale a “barramento ocupado” – por isso o nome evita esse rótulo por padrão.

2.3 Entrada do botão e antirrepique

O GPIO 17 é configurado como entrada com resistor interno de redução (*pull-down*). Dessa forma, o terminal permanece em nível baixo com o botão aberto e passa ao nível alto quando o contato conecta o GPIO à alimentação de 3,3 [V].

Essa configuração dispensa a implementação física de um resistor de *pull-down* externo: o próprio microcontrolador disponibiliza esse resistor internamente, ativado por software (`Pin.PULL_DOWN`), o que simplifica o circuito, reduz o número de componentes e elimina uma fonte de erro de montagem.

Os dois botões de controle de velocidade (GPIO 34 e GPIO 35, Tabela 1) não podem usar essa mesma facilidade: são pinos exclusivamente de entrada e não dispõem de nenhum resistor interno de redução ou de elevação (*pull-up*). Por isso, cada um desses dois botões utiliza um resistor externo de *pull-down* de 10 [kΩ], já incluído em `diagram.json`, reproduzindo o mesmo comportamento solto = LOW / pressionado = HIGH do botão principal.

Não foi acrescentado filtro RC externo em nenhum dos três botões. O antirrepique (*bouncing*) é realizado por programa, sem bloqueio: o nível é amostrado periodicamente e uma alteração somente é aceita quando permanece estável por aproximadamente 30 [ms]. Essa estratégia é suficiente para a aplicação e evita oscilações do LED verde e atualizações repetidas do OLED.

Essas duas medidas fazem sentido sobretudo para uma implementação física: um botão pulsador mecânico real produz múltiplas transições elétricas espúrias durante o acionamento, e é esse ruído que o antirrepique elimina. O simulador Wokwi, por outro lado, modela o botão como um interruptor ideal, sem esse ruído de contato. Na simulação, portanto, o antirrepique não é estritamente necessário e é mantido apenas como boa prática, de modo que o mesmo código já esteja correto caso o projeto seja migrado para um botão físico sem qualquer alteração de lógica.

2.4 Interface dos três mostradores

Os três mostradores do sistema usam duas interfaces de comunicação diferentes, cada uma escolhida pela natureza do tráfego que efetivamente precisa suportar.

Os dois OLEDs SSD1306 se comunicam por I²C, cada um em seu próprio barramento de hardware independente (`machine.I2C(0)` e `machine.I2C(1)`), de modo que ambos podem ser endereçados ao mesmo tempo sem disputa de barramento. O primeiro usa GPIO 32 como relógio (SCL) e GPIO 16 como dados (SDA); o segundo usa GPIO 15 (SCL) e GPIO 22 (SDA). O endereço adotado para os dois é 0x3C, com resolução de 128 por 64 [pixels] cada. A interface I²C é um requisito fixo do projeto, declarado explicitamente no código, no diagrama e na documentação, e aplicado igualmente aos dois mostradores monocromáticos. Ela usa apenas dois sinais de comunicação por barramento e é adequada às atualizações periódicas, porém pouco frequentes (a cada 250 [ms]), dos dois gráficos de uso de CPU e RAM (Seção 3.7).

A tela TFT ILI9341, por sua vez, usa SPI genuíno de 4 fios – relógio (SCK), dados de saída (MOSI), seleção de circuito (CS) e dado/comando (D/C), além de uma linha independente de reinicialização (RST) –, em GPIO 18, 23, 5, 21 e 19 respectivamente, com resolução de 240 por 320 [pixels] e cor de 16 bits (RGB565). Diferentemente dos dois OLEDs, a TFT funciona como um console de registro (*log*) colorido (Seção 3.8) que recebe uma nova linha de texto a cada evento relevante do sistema – uma frequência de atualização bem maior e mais irregular que a dos dois gráficos de uso de CPU/RAM. SPI oferece maior taxa de transferência que I²C à custa de mais sinais de controle, o que se justifica aqui pelo volume de dados que esse console efetivamente movimenta.

3 Desenvolvimento do programa main.py

3.1 Estrutura geral

O arquivo main.py concentra a inicialização do hardware e a coordenação das tarefas.

Uma organização representativa da aplicação é mostrada a seguir:

Listagem 1: Estrutura lógica resumida do programa.

```
1 BLINKING_LEDS = [  
2     [red_led, RED_LED_BLINK_INTERVAL_MS],  
3     [blue_led, BLUE_LED_BLINK_INTERVAL_MS],  
4     [yellow_led, YELLOW_LED_BLINK_INTERVAL_MS],  
5     [white_led, WHITE_LED_BLINK_INTERVAL_MS],  
6     [orange_led, ORANGE_LED_BLINK_INTERVAL_MS],  
7     [red_led_2, RED_LED_2_BLINK_INTERVAL_MS],  
8 ]  
9  
10 async def blink_led(entry):  
11     """Toggle one LED at its own interval, independent of every other task."""  
12     led, _ = entry  
13     while True:  
14         led.value(not led.value())  
15         await asyncio.sleep_ms(entry[1])  
16  
17 async def monitor_button(initial_state):  
18     """React to every debounced push-button transition."""  
19     await debounce_button(push_button, initial_state, apply_button_state)  
20  
21 async def main():  
22     initial_button_state = bool(push_button.value())  
23     apply_button_state(initial_button_state)  
24  
25     for entry in BLINKING_LEDS:  
26         asyncio.create_task(blink_led(entry))  
27     asyncio.create_task(scheduler_idle_task())  
28     asyncio.create_task(update_cpu_graph())  
29     asyncio.create_task(update_ram_graph())  
30     asyncio.create_task(print_status())  
31     asyncio.create_task(  
32         monitor_step_button(decrease_speed_button, lambda: apply_blink_speed_step(-1))  
33     )  
34     asyncio.create_task(  
35         monitor_step_button(increase_speed_button, lambda: apply_blink_speed_step(+1))  
36     )  
37     await monitor_button(initial_button_state)
```

Este trecho é uma síntese da arquitetura; o código completo e executável está disponível no repositório. Onze dessas corrotinas – as seis tarefas de LED, `scheduler_idle_task()`, as duas tarefas de mostrador e as duas dos botões de velocidade – são despachadas com `asyncio.create_task()` dentro de `main()` e passam a rodar de forma independente a partir desse ponto. O monitoramento do botão principal é tratado de forma diferente: `main()` termina com `await monitor_button(initial_button_state)` em

vez de mais um `create_task()`. Como `monitor_button()` roda em laço infinito e nunca retorna, `main()` nunca chega a terminar sua própria execução – ela efetivamente passa a *ser* o monitoramento do botão principal a partir dali, e não uma tarefa separada dele. Sob o escalonador cooperativo, isso não distingue esse fluxo dos demais em termos de comportamento observável: `main()` continua cedendo o controle nos mesmos pontos de espera que qualquer outra tarefa.

3.2 Assincronismo cooperativo

A biblioteca `asyncio` do MicroPython foi escolhida como arquitetura exclusiva do projeto. Em vez de um superlaço (*superloop*) que verifica manualmente diversos temporizadores, cada função independente pode ser organizada como uma corrotina (*coroutine*). A instrução `await asyncio.sleep_ms(...)` suspende apenas a tarefa atual e devolve o controle ao escalonador (*scheduler*).

Essa decisão foi tomada pelos seguintes motivos:

- Separação clara entre o piscar de cada LED, a leitura de cada botão e a atualização de cada mostrador;
- Menor acoplamento entre funcionalidades e facilidade de manutenção e escalabilidade;
- Prevenção de congelamentos indefinidos causados por `time.sleep()` no fluxo principal, já que `await asyncio.sleep_ms(...)` devolve o controle ao escalonador em vez de travar a CPU.

3.3 O que o escalonamento cooperativo garante – e o que não garante

O `asyncio` do MicroPython roda em um único núcleo, sem preempção e sem paralelismo real: a qualquer instante, no máximo uma corrotina está de fato executando; todas as demais estão suspensas, aguardando sua vez. "Independente" e "concorrente", nas seções deste relatório, significam independência *lógica* – nenhuma tarefa chama ou espera diretamente por outra – não isolamento de tempo de execução.

Especificamente:

- **Escritas nos mostradores bloqueiam o escalonador inteiro.** `show()`, `fill_rect()` e `text()` (SSD1306 e ILI9341) são chamadas síncronas, sem nenhum ponto `await` interno: enquanto uma delas está em andamento, nenhuma outra tarefa – LED piscante, botão ou outro mostrador – pode ser retomada, por mais tempo que essa escrita leve. O `asyncio` evita que um `time.sleep()` explícito trave o programa indefinidamente, mas não transforma essas transferências I²C/SPI em operações não bloqueantes.
- **Os intervalos de 250 [ms] dos dois OLEDs são um piso, não um período exato.** `await asyncio.sleep_ms(250)` garante apenas que pelo menos 250 [ms] se passem antes da retomada; se o restante do ciclo (leitura do sensor, desenho, escrita bloqueante) levar tempo adicional, a cadência real observada é maior que 250 [ms] e pode variar de amostra para amostra – ver a medição explícita por `time.ticks_us()` em `update_cpu_graph()` (Seção 3.7).
- **Uma pressão de botão pode não ser amostrada.** O botão principal e os dois de velocidade são lidos a cada 5 [ms] (Seção 2.3), mas essa amostragem só ocorre quando a tarefa de monitoramento correspondente é retomada. Se uma escrita bloqueante em algum mostrador estiver em andamento durante uma pressão inteira – início e fim do contato dentro dessa mesma janela –, nenhuma amostra intermediária existe para detectá-la, e a transição pode não ser registrada.

Essas limitações não invalidam a arquitetura: para os intervalos de LED (centenas de milissegundos a segundos) e para uma aplicação interativa, não crítica em tempo real, os atrasos introduzidos por uma única escrita de mostrador (tipicamente frações de milissegundo a poucos milissegundos) são pequenos e

normalmente imperceptíveis. Eles são, porém, reais, mensuráveis e vistos diretamente nos LEDs laranja e amarelo (Seção 3.6) – e são a causa raiz da dessincronização dos LEDs azuis e da eventual necessidade de segurar o botão por mais tempo, ambas já descritas nesta seção.

O assincronismo adotado é cooperativo. Portanto, as tarefas devem alcançar pontos de espera com frequência e evitar rotinas longas ou bloqueantes.

3.4 LEDs piscantes e controle de velocidade

O sistema controla seis LEDs piscantes independentes (Tabela 1), todos fisicamente azuis no circuito atual. Em vez de uma corrotina escrita à mão para cada LED, `main.py` mantém uma única lista, `BLINKING_LEDS`, com um par [objeto Pin, intervalo em milissegundos] por LED, e lança uma tarefa `blink_led()` idêntica para cada par dessa lista, em um laço `for` (Listagem 1). Cada par é uma lista mutável, não uma tupla, porque o intervalo é reescrito em tempo real pelos botões de velocidade enquanto a tarefa correspondente já está em execução: `blink_led()` relê `entry[1]` a cada ciclo, em vez de capturá-lo uma única vez no início, permitindo que uma mudança de velocidade tenha efeito imediato sem reiniciar nenhuma tarefa.

Os dois botões de controle de velocidade (GPIO 34 diminui, GPIO 35 aumenta – Seção 2.3) alteram a velocidade dos seis LEDs de uma só vez: cada pressão desloca, em uma unidade, um fator de escala em potência de dois aplicado sobre o intervalo-base de 500 [ms] de cada LED, limitado a um intervalo mínimo de 125 [ms] e máximo de 4 [s]. A cada mudança, o novo intervalo é registrado no console da TFT em azul (Seção 3.8).

3.5 Botão principal, LED verde e console de registro

O botão principal (GPIO 17) e os dois botões de velocidade compartilham a mesma corrotina genérica de antirrepique, `debounce_button()`: ela amostra o pino periodicamente, aguarda 30 [ms] de estabilidade (Seção 2.3) e só então invoca uma função de retorno (*callback*) específica para cada botão. Para o botão principal, essa função é `apply_button_state()`: liga ou desliga o LED verde (GPIO 4) conforme o novo estado estável e registra a transição no console da TFT, em verde.

O console de registro da TFT é o único registro dedicado a esse evento: nenhum OLED exibe mensagem de texto associada ao botão, já que os dois OLEDs são gráficos de uso de CPU e RAM (Seção 3.7) sem espaço livre para texto adicional. Toda mensagem passada a `console_log()` – inclusive esta – também é sempre impressa no console serial, além de (quando presente) escrita na TFT (Seção 3.8 explica por que essa duplicação é incondicional, não uma detecção de ausência da TFT).

3.6 Indicadores de atividade: barramento e escalonador

Dois LEDs adicionais tornam visível, em tempo real, um comportamento interno do escalonador cooperativo que de outra forma não teria nenhuma manifestação observável no circuito.

O LED laranja (GPIO 13, `bus_idle_led`) permanece aceso por padrão, indicando que nenhum dos três mostradores está sendo escrito naquele instante, e apaga apenas durante uma escrita bloqueante em algum deles – lido ao contrário, apagado equivale a “barramento ocupado”. Um par de funções auxiliares, `_bus_busy_begin()` e `_bus_busy_end()`, encapsula toda chamada bloqueante de `show()/fill_rect()/text()` nos três mostradores: apaga o LED no início da chamada, mede sua duração real com `time.ticks_us()` e o acende de volta ao final – acumulando essa mesma duração em `bus_busy_time_us`, consumida pelo gráfico de uso de CPU (Seção 3.7). O nome `bus_idle_led` (e o rótulo “barramento ocioso” usado neste relatório) descreve seu estado *padrão*, não um evento; o próprio `diagram.json` rotula o LED no esquemático com a mesma ressalva de polaridade.

O LED amarelo (GPIO 2, `scheduler_idle_led`) liga e desliga em uma tarefa própria, `scheduler_idle_task()`, sem nenhum compromisso de tempo fixo: a cada iteração ela apenas alterna o LED e devolve o controle ao escalonador com `await asyncio.sleep_ms(0)`. Essa chamada cede o controle e recoloca a tarefa na fila para ser escalonada de novo imediatamente – ela *não* reduz a prioridade da tarefa, porque o `asyncio` do MicroPython não implementa níveis de prioridade: essa tarefa concorre pela mesma vez de execução em rodízio (*round-robin*) que qualquer outra tarefa pronta, não apenas nos intervalos ociosos entre elas. O LED, portanto, não é um indicador literal de “nada mais tem trabalho pendente” nem uma implementação de uma tarefa de prioridade mínima (*idle task*) de sistema operacional de tempo real – é uma visualização grosseira de atividade e latência do escalonador: com que frequência essa tarefa específica consegue retomar a execução. Uma vez por segundo, a mesma tarefa registra no console da TFT, em amarelo, quantas iterações desse laço conseguiu completar no último segundo. Esse número serve como medida comparativa de vazão entre execuções ou momentos diferentes (mais iterações/s = o restante do sistema cedeu o controle com mais frequência ou por períodos mais curtos) – não como um percentual calibrado de ociosidade ou de capacidade livre, já que nenhum valor de referência de “100% ocioso” foi medido para normalizá-lo.

3.7 Mostradores de uso de CPU e RAM (dois OLEDs)

Os dois OLEDs SSD1306 funcionam como gráficos de barras rolantes, no estilo de um monitor de tarefas, atualizados a cada 250 [ms] (`update_cpu_graph()` e `update_ram_graph()`). Cada amostra ocupa uma coluna de pixels na largura do OLED (até 128 amostras de histórico); ao atingir o limite, a amostra mais antiga é descartada e todas as colunas são redesenhadas a cada janela.

O primeiro OLED plota um rótulo chamado “CPU”⁴, mas o valor efetivamente medido é mais estreito do que esse nome sugere: é a fração de cada janela de amostragem ocupada pelas chamadas síncronas instrumentadas de desenho e transferência dos três mostradores – `show()`, `fill_rect()` e `text()`, cronometradas por inteiro entre `_bus_busy_begin()` e `_bus_busy_end()` (Seção 3.6) e acumuladas em `bus_busy_time_us`. Como o cronômetro envolve a chamada inteira, e não apenas a transferência física, o valor inclui tanto o trabalho de *framebuffer* e desenho feito em Python (por exemplo, o laço de até 128 colunas em `draw_usage_graph()`, ou a conversão de glifo para pixels em `ili9341.py`) quanto a própria transferência I²C/SPI – as duas coisas juntas, não isoladamente uma ou outra. Isso não é a única fonte de uso de CPU da aplicação: amostragem e antirrepique dos três botões, a contabilidade de `scheduler_idle_task()`, a formatação da linha impressa por `print_status()` e o restante do código Python também consomem tempo de processador e não entram nessa soma. O gráfico é, portanto, um indicador parcial e aproximado – a fração de tempo gasta especificamente dentro das escritas de mostrador instrumentadas – não uma medição completa de utilização de CPU. A janela de amostragem em si é medida com `time.ticks_us()`, não assumida como exatamente 250 [ms] (Seção 3.3): `asyncio.sleep_ms()` garante apenas um atraso mínimo, e uma chamada mais lenta a `console_log()` em outra tarefa pode empurrar o intervalo real bem além do solicitado – dividir pelo intervalo assumido, quando mais curto que o real, foi a causa de uma leitura defeituosa acima de 100% observada durante os testes e desde então corrigida.

O segundo OLED plota a fração ocupada do heap gerenciado pelo coletor de lixo do MicroPython (`gc.mem_alloc()` / `gc.mem_free()`), em seu próprio barramento I²C independente (Seção 2.4). Esse valor não é a RAM física total do ESP32: exclui, entre outros, a pilha (*stack*) de execução, alocações nativas/C internas ao firmware e qualquer memória reservada fora do heap gerenciado pelo coletor de lixo – é uma medida real, porém parcial, de pressão de memória sobre esse heap especificamente.

⁴O rótulo CPU foi mantido, tanto no código quanto neste relatório, por já estar consolidado no console serial (`print_status()`) e no esquema de cores do console da TFT (Seção 3.8), e por caber no espaço reduzido de um mostrador de 128 por 64 [pixels] monocromático; a alternativa seria substituí-lo por um nome tecnicamente mais preciso, como `DISPLAY` ou

`SYNC`, em troca de exigir uma mudança coordenada em `main.py`, no console da TFT e em toda a documentação. Optou-se por manter o rótulo curto e qualificar seu significado explicitamente neste parágrafo, em vez de renomeá-lo.

3.8 Console de registro colorido (TFT)

A tela TFT funciona como um console de registro (*log*) rolante, no estilo do `dmesg` de um sistema Unix ou do histórico de instalação de um gerenciador de pacotes: cada evento do sistema imprime uma linha de texto colorida; ao preencher a última linha visível, a próxima linha volta para o topo da tela (não há rolagem real de conteúdo – ver `console_log()` em `main.py`). Cada cor está associada a um subsistema e, sempre que esse subsistema tem um LED físico correspondente, à mesma cor desse LED:

- Azul – os seis LEDs piscantes;
- Verde – o botão principal e o LED verde;
- Laranja – o LED indicador de barramento ocioso;
- Amarelo – o LED indicador de escalonador ocioso;
- Vermelho – o gráfico de uso de CPU;
- Roxo/lilás – o gráfico de uso de RAM;
- Branco – mensagens gerais, sem um subsistema colorido correspondente (inicialização, diagnósticos de falha).

A ligação SPI da TFT neste projeto é somente de escrita: usa SCK, MOSI, CS e D/C (Seção 2.4), mas não MISO, e `ili9341.py` nunca lê de volta um identificador do painel. Por isso, `create_tft_display()` só retorna `None` quando a própria construção do objeto Python levanta `OSError` – uma falha de driver ou de configuração de pino –, não quando o painel físico simplesmente não está conectado: nesse segundo caso, as escritas SPI muito provavelmente são aceitas sem erro algum, e `tft_display` permanece um objeto válido mesmo sem nenhuma tela real do outro lado do barramento. Como essa detecção de ausência não é confiável, `console_log()` não depende dela: toda linha é sempre impressa no console serial, incondicionalmente, além de (quando `tft_display` não é `None`) escrita na TFT. Essa duplicação incondicional, em vez de restrita ao caso `tft_display is None`, é o que garante que uma TFT fisicamente ausente mas eletricamente silenciosa – que não gera `OSError` algum – ainda assim não silencie nenhum evento do console.

A implementação de texto (`ili9341.py`, método `text()`) foi otimizada após um levantamento de desempenho: a versão original convertia cada glifo em pixels individuais por meio de `framebuf.FrameBuffer.pixel()`, chamada até cerca de 1920 vezes por linha de texto. A versão atual pré-calcula, uma única vez por combinação de cor de texto e cor de fundo, uma tabela de conversão de 256 entradas – uma por valor de byte possível – para os 16 bytes RGB565 correspondentes a essa fatia de 8 pixels do glifo, e reutiliza essa tabela a cada caractere renderizado, eliminando o laço pixel a pixel do caminho crítico. Do mesmo modo, `console_log()` evita limpar a linha inteira do console a cada mensagem: apenas o trecho da linha além da largura real do texto é apagado (`fill_rect()`), já que `text()` já pinta sua própria cor de fundo sob cada caractere que desenha.

4 Estratégia de verificação e testes isolados

O desenvolvimento foi acompanhado por testes progressivos. A separação dos subsistemas permitiu distinguir erros de ligação, inicialização e lógica de aplicação.

1. **Teste isolado do primeiro LED piscante:** Utilizou um laço bloqueante mínimo somente para diagnóstico. O GPIO 26 foi alternado a cada 500 [ms] e o estado foi registrado no monitor serial. O objetivo foi confirmar placa, pino, resistor, polaridade e retorno ao terra sem interferência das demais funções.
2. **Teste do mesmo LED com o idioma de alternância:** Repetiu o teste anterior usando `led.value(not led.value())`, o mesmo idioma de alternância de nível que `blink_led()` efetivamente usa em `main.py`, ainda em um laço bloqueante, sem `asyncio` – isolando essa mudança de estilo de código como uma variável independente da introdução do `asyncio` no teste seguinte.
3. **Teste do mesmo LED com asyncio:** Repetiu o teste anterior, agora encapsulado em uma corrotina agendada com `asyncio.create_task()` dentro de `asyncio.run()`, isolando especificamente o suporte a `asyncio` do firmware como variável, separado da fiação e do idioma de alternância já confirmados.
4. **Teste isolado do botão e do LED verde:** Amostrou o GPIO 17 periodicamente e refletiu o estado lido no GPIO 4, sem qualquer dependência de nenhum LED piscante ou do OLED. O objetivo foi confirmar a fiação do botão, o resistor interno de redução e o comportamento ativo em nível alto isoladamente.
5. **Teste básico do OLED:** Inicializou o barramento I²C em GPIO 32 e GPIO 16, confirmou a detecção do endereço 0x3C por meio de `i2c.scan()` e apresentou uma mensagem simples.
6. **Diagnóstico gráfico do SSD1306:** Exercitou todos os pixels ligados e desligados, padrões quadriculados complementares, varreduras horizontais e verticais, grade, pixels individuais, linhas, retângulos, áreas preenchidas, texto, inversão, contraste, desligamento, religamento e deslocamento do quadro.
7. **Teste básico dos LEDs extras:** Verificou, um de cada vez e com um laço bloqueante mínimo, a fiação dos cinco LEDs piscantes adicionais (GPIO 14, 27, 25, 33 e 12), no mesmo espírito do teste 1.
8. **Seis LEDs piscantes concorrentes:** Reuniu o LED do teste 1–3 e os cinco LEDs do teste 7 rodando ao mesmo tempo, cada um em sua própria tarefa `asyncio`, exatamente como `BLINKING_LEDS` em `main.py`.
9. **Botões de velocidade:** Validou os dois botões pulsadores adicionais (GPIO 34 e GPIO 35) e seus resistores externos de *pull-down*, diferentemente do resistor interno usado pelo botão principal no teste 4.
10. **LEDs indicadores:** Validou apenas a fiação dos dois LEDs indicadores de estado do sistema – laranja de barramento ocioso (GPIO 13) e amarelo de atividade do escalonador (GPIO 2) –, alternando-os em um temporizador fixo; não exercita a lógica real de ocupação/atividade de `main.py`, que depende do barramento e do escalonador em funcionamento para ter sentido.
11. **Segundo OLED:** Validou o segundo barramento I²C independente (I2C(1), GPIO 15/22) do segundo OLED, isolado do barramento do primeiro.
12. **TFT básica:** Validou a fiação SPI de 4 fios da TFT (SCK 18, MOSI 23, CS 5, D/C 21) e sua linha de reinicialização (RST 19): inicialização do painel e preenchimentos de cor sólida.
13. **Diagnóstico de texto da TFT:** Exercitou o método `text()` de `ili9341.py` – a primitiva sobre a qual o console de registro de `main.py` é construído – em todas as cores usadas pelo console, incluindo o retorno ao topo da tela ao preencher todas as linhas visíveis.
14. **Reserva do console para o serial (sem hardware):** Único teste puramente lógico da sequência – não toca nenhum Pin/I²C/SPI e roda em CPython comum, sem Wokwi nem placa. Reproduz a lógica

de `console_log()` para confirmar que toda mensagem chega ao console serial tanto com `tft_display` igual a `None` quanto com um objeto ativo simulando uma TFT fisicamente ausente porém eletricamente silenciosa (Seção 3.8).

A sequência de testes reduziu o espaço de busca de falhas: se um subsistema simples não funcionasse, o problema poderia ser tratado antes da integração com o restante da aplicação. A tabela de dependências completa entre os quatorze testes, e uma recomendação de subconjunto mínimo (*smoke test*) para quando não há tempo de rodar todos, estão em `tests/README.md`.

5 Circuito simulado

A Figura 1 apresenta o estado atual do circuito no Wokwi. Com o número maior de componentes e sinais, as cores dos condutores no diagrama deixaram de seguir uma convenção única por função; a pinagem completa e atual de todo o projeto está na Tabela 1 (Seção 2).

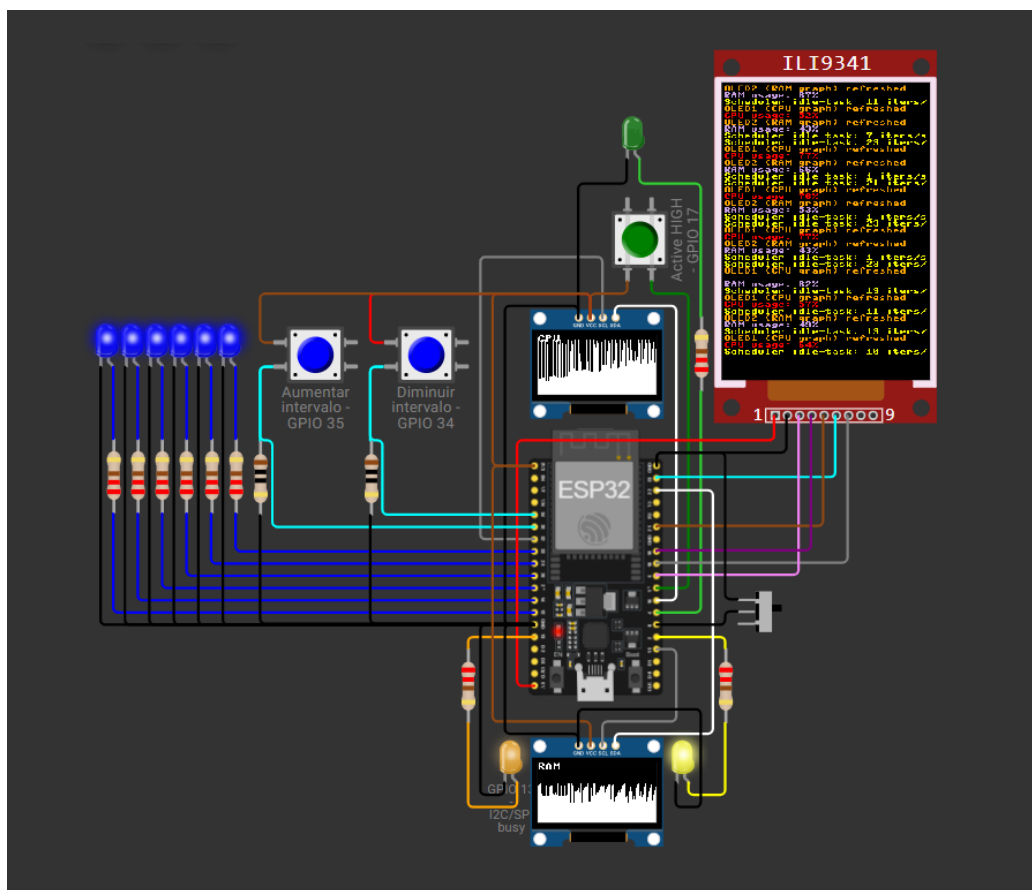


Figura 1: Circuito do projeto no Wokwi: ESP32-DevKitC V4, dois OLEDs SSD1306, display TFT ILI9341, seis LEDs azuis, um LED verde, dois LEDs indicadores de estado (laranja e amarelo), três botões pulsadores e uma chave deslizante.

6 Conclusão

O projeto atende aos requisitos definidos por meio de uma implementação modular e reproduzível. A escolha da ESP32-DevKitC V4 reduz ambiguidades de pinagem, enquanto o uso explícito de cada GPIO (Tabela 1) preserva a correspondência entre código, documentação e circuito. As duas interfaces de

comunicação adotadas – I²C para os dois OLEDs e SPI de 4 fios para a TFT – atendem adequadamente à natureza distinta do tráfego de cada mostrador, conforme discutido na Seção 2.4.

A arquitetura assíncrona permite a concorrência de treze fluxos independentes (Seção 3.1) sob um único escalonador cooperativo. O antirrepique compartilhado elimina a necessidade de filtro externo nesta aplicação e em suas três variantes de botão, e as otimizações aplicadas ao console de registro da TFT – tabela de conversão de cor pré-calculada, limpeza apenas do trecho residual de cada linha – mantêm o tempo de escrita bloqueante baixo o suficiente para não dominar o próprio gráfico de uso de CPU que ele ajuda a alimentar. Por fim, os quatorze testes isolados e progressivos (Seção 4) forneceram evidências de funcionamento de cada subsistema antes da integração completa.

A entrega é complementada por dois meios de acesso: a simulação pública no Wokwi e o repositório versionado no GitHub. Em conjunto, eles possibilitam executar, inspecionar, reproduzir e evoluir o projeto.

Referências eletrônicas

- Projeto no Wokwi: <https://wokwi.com/projects/471528241540407297>
- Repositório no GitHub: <https://github.com/Argus-Kepheus/esp32-asyncio>
- Guia da ESP32-DevKitC V4: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html
- MicroPython para ESP32: <https://docs.micropython.org/en/latest/esp32/quickref.html>
- Documentação da asyncio: <https://docs.micropython.org/en/latest/library/asyncio.html>
- Componente SSD1306 do Wokwi: <https://docs.wokwi.com/parts/board-ssd1306>